

Introduction to MATLAB

What is MATLAB?

- ❖ MATLAB (an abbreviation of "MATrix LABoratory") is a **multi-paradigm programming language** and numeric computing environment developed by MathWorks.
- ❖ MATLAB allows **matrix manipulations, plotting of functions and data, implementation of algorithms, creation of user interfaces**, and interfacing with programs written in other languages.

How to start MATLAB?

One can enter MATLAB by double-clicking on the MATLAB shortcut icon on Windows desktop . When MATLAB is started, a special window called the MATLAB desktop appears which consists of following sub-windows:

- ❖ Command Window- This is the main part of the window where commands can be entered. It is indicated by the command prompt (`>>`).
- ❖ Current Folder- The files created in MATLAB are saved in current directory. The saved files can be viewed and accessed for further use.
- ❖ Workspace- The workspace shows all the variables created and/or imported from files and displays type and size of the variables.
- ❖ Command History- The commands typed in the command window automatically get recorded and stored in command history day wise. These can be retrieved any time for execution. The command history gets cleaned only when the command 'Clear Command History' is used.

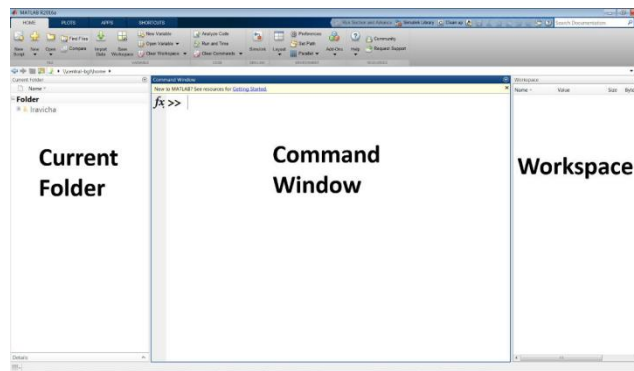


Figure MATLAB Desktop

How to open and save files?

In the toolbar of MATLAB window, the options 'HOME' or 'EDITOR/LIVE EDITOR' possess the icon  which is to 'open' a file with '.m' or '.mlx' extension. The particular file gets loaded in editor window once it is clicked on. All DOS commands are also applicable. A click on the icon  saves the file.

Desktop management

Function	Example	Description
<code>save</code>	<code>save data.mat</code>	Save your current workspace to a MAT-file.
<code>load</code>	<code>load data.mat</code>	Load the variables in a MAT-file to the Workspace.
<code>clear</code>	<code>clear</code>	Clear all variables from the Workspace.
<code>clc</code>	<code>clc</code>	Clear all text from the Command Window.
<code>format</code>	<code>format long</code>	Change how numeric output is displayed.

Arithmetic Operations:

- ❖ Understand how to use MATLAB as a calculator to carry out arithmetic operations such as addition, subtraction, multiplication, division and exponentiation.
- ❖ Use format commands to control floating point output display.
- ❖ Execute arithmetic expressions involving various operations.

Arithmetic Operators and Special Characters

Operations	Symbol	Examples	Operations	Symbol	Examples
Addition	+	$4 + 3 = 7$	Right division	/	$1/2$
Subtraction	-	$5 - 3 = 2$	Left division	\	$4/8 = 8/4 = 2$
Multiplication	*	$6 * 7 = 42$	Exponentiation	^	$5^3 = 125$

Some mathematical symbols:

1. π - π 2. Inf - ∞ 3. i (or j) - $\sqrt{-1}$

Array Operations:

Array operations work on corresponding elements of arrays with equal dimensions. Each element in the first operand gets matched up with the element in the same location in the second operand. The following table provides a summary of arithmetic array operators in MATLAB.

Arithmetic array operators

Operator	Purpose	Description
+	Addition	$A + B$ adds A and B
+	Unary plus	$+A$ returns A
-	Subtraction	$A - B$ subtracts B from A
-	Unary minus	$-A$ negates the elements of A
.*	Element-wise	$A .* B$ is the element-by-element product of A and B
.^	Element-wise power	$A.^B$ is the matrix with elements $A(i, j)$ to the power $B(i, j)$
./	Right array division	$A ./ B$ is the matrix with elements $A(i, j)/B(i, j)$
.'	Array transpose	$A.'$ is the array transpose of A.

Array types

Example	Description
4	scalar
[3 5]	row vector
[1;3]	column vector
[3 4 5;6 7 8]	matrix

Evenly-spaced vectors

Example	Description
1:4	Create a vector from 1 to 4, spaced by 1, using the colon(:) operator.
1:0.5:4	Create a vector from 1 to 4, spaced by 0.5.
linspace (1,10,5)	Create a vector with 5 elements. The values are evenly spaced from 1 to 10.

Creating matrices

Example	Description
rand (2)	Create a square matrix with 2 rows and 2 columns.
zeros (2,3)	Create a rectangular matrix with 2 rows and 3 columns.

Indexing

Example	Description
A(end ,2)	Access the element in the second column of the last row.
A(2,:)	Access the entire second row
A(1:3,:)	Access all columns of the first three rows.
A(2) = 11	Change the value of the second element an array to 11.

Array operations

Example	Description
<pre>[1 1; 1 1]*[2 2;2 2] ans = 4 4 4 4</pre>	Perform matrix multiplication .
<pre>[1 1; 1 1].*[2 2;2 2] ans = 2 2 2 2</pre>	Perform element-wise multiplication .

PROGRAM 1: Solution of first order differential equation and plotting the solution curves

Example 1: Solve the first order differential equation $dy/dx = xy$ and plot the graph.

MATLAB code:

```
>> syms y(x);  
>> ode=diff(y,x)==x*y;  
>> sol=dsolve(ode)
```

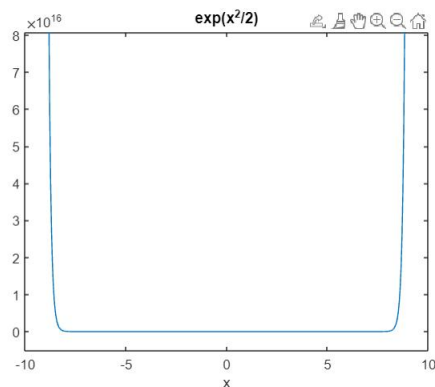
Output

sol =

$C1 \cdot \exp(x^2/2)$

Syntax to plot the graph:

```
>> ezplot(exp(x^2/2), [-10,10])
```



Example 2: Solve $\frac{dy}{dx} = y^2 + \frac{y}{x}$

MATLAB code:

```
>> syms y(x);  
>> ode=diff(y,x)==(y^2+(y/x));  
>> sol=dsolve(ode)
```

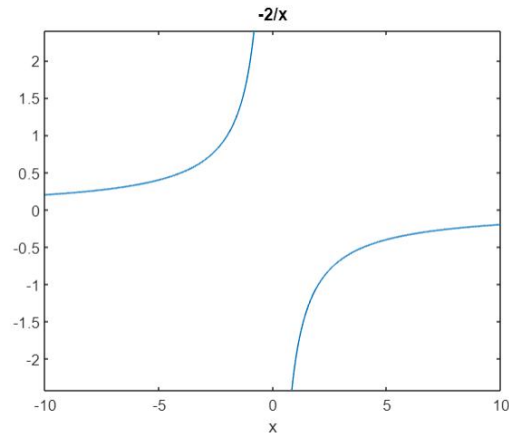
Output

sol =

$\frac{x}{(-x^2/2 + C1)}$
0

Syntax to plot the graph :

```
>> ezplot(x/(-x^2/2), [-10,10])
```



PROGRAM 2: Solution of differential equations with initial conditions.

Example 1 -Solve the second-order differential equation $\frac{d^2y}{dt^2} = ay$ with the initial conditions $y(0)=0$ and $y'(0)=1$..

MATLAB code:

```
syms y(t) a
eqn = diff(y,t,2) == a*y;
Dy = diff(y,t);
cond = [y(0)==0, Dy(0)==1];
ySol(t) = dsolve(eqn,cond)
```

Output:

```
ySol(t) =
(exp(-a^(1/2)*t)*(exp(2*a^(1/2)*t) - 1))/(2*a^(1/2))
```

Example 2: Solve the second-order differential equation $\frac{d^2y}{dt^2} = a^2y$ with the initial conditions $y(0)=b$ and $y'(0)=1$.

MATLAB code:

```
syms y(t) a b
eqn = diff(y,t,2) == a^2*y;
Dy = diff(y,t);
cond = [y(0)==b, Dy(0)==1];
ySol(t) = dsolve(eqn,cond)
```

Output:

```
ySol(t) =
(exp(a*t)*(1 + a*b))/(2*a) + (exp(-a*t)*(- 1 + a*b))/(2*a)
```

PROGRAM 3: Solution of homogeneous ordinary differential equations of higher order.

Example 1: Solve the second order homogeneous differential equation. $2x^2 \frac{d^2 y}{dx^2} + 3x \frac{dy}{dx} - y = 0$.

MATLAB code:

```
syms y(x)
ode = 2*x^2*diff(y,x,2)+3*x*diff(y,x)-y == 0;
ySol(x) = dsolve(ode
```

Output:

```
ySol(x) =
C1/(3*x) + C2*x^(1/2)
```

Example 2: Solve the third-order differential equation $\frac{d^3 u}{dx^3} = u$ with three initial conditions. $u(0) = 1$, $u'(0) = -1$, $u''(0) = \pi$.

MATLAB code:

```
syms u(x)
Du = diff(u,x);
D2u = diff(u,x,2);
ode = diff(u,x,3) == u;
cond1 = u(0) == 1;
cond2 = Du(0) == -1;
cond3 = D2u(0) == pi;
conds = [cond1 cond2 cond3];
uSol(x) = dsolve(ode,conds)
```

Output:

```
uSol(x) =
(pi*exp(x))/3 - exp(-x/2)*cos((3^(1/2)*x)/2)*(pi/3 - 1) - (3^(1/2)*exp(-x/2)*sin((3^(1/2)*x)/2)*(pi + 1))/3
```

PROGRAM 4: Solution of non-homogeneous ordinary differential equations of higher order.

Example 1: Solve the second-order differential equation with two initial conditions:

$$d^2y/dx^2 = \cos(2x) - y, \quad y(0) = 1, \quad y'(0) = 0.$$

MATLAB code:

```
syms y(x)
Dy = diff(y);
ode = diff(y,x,2) == cos(2*x)-y;
cond1 = y(0) == 1;
cond2 = Dy(0) == 0;
conds = [cond1 cond2];
ySol(x) = dsolve(ode,conds);
ySol = simplify(ySol)
```

Output:

```
ySol(x) =
1 - (8*sin(x/2)^4)/3
```

Example 2: Solve the second-order differential equation (Airy equation) $d^2y/dx^2 = xy$.

MATLAB code:

```
syms y(x)
ode = diff(y,x,2) == x*y;
ySol(x) = dsolve(ode)
```

Output:

```
ySol(x) =
C1*airy(0,x) + C2*airy(2,x)
```

.....

PROGRAM 5: Finding the largest eigen value by Rayleigh power method.

Example 1: Write and execute MATLAB program to find the dominant eigen value and the corresponding eigen vector by power method $A = \begin{bmatrix} 4 & 1 & -1 \\ 2 & 3 & -1 \\ -2 & 1 & 5 \end{bmatrix}$. Perform 4 iterations, taking initial guess as $[1,0,0]^T$.

MATLAB code:

```
clc;
disp(' Matrix ');
A=[4 1 -1;2 3 -1;-2 1 5]

disp('Initial guess');
v=[1;0;0]
disp('Number of iterations');
n=6
v0=v;
for i=1:n
v=A*v0;
M=max(v);
v=v/M;
v0=v;
fprintf('iteration number # %d\n',i)
fprintf('current eigen value is %.4f\n', M)
fprintf('current eigen value is ')
v
end
```

OUTPUT:

Matrix

```
A =
     4     1    -1
     2     3    -1
    -2     1     5
```

Initial guess

```
v =
     1
     0
     0
```

Number of iterations

```
n = 4
iteration number # 1
current eigen value is 4.0000
current eigen value is
v =
    1.0000
    0.5000
   -0.5000
```



```

iteration number # 2
current eigen value is 5.0000
current eigen value is
v =
    1.0000
    0.8000
   -0.8000

```

```

iteration number # 3
current eigen value is 5.6000
current eigen value is
v =
    1.0000
    0.9286
   -0.9286

```

```

iteration number # 4
current eigen value is 5.8571
current eigen value is
v =
    1.0000
    0.9756
   -0.9756

```

Example 2: Write and execute MATLAB program to find the dominant eigen value and the corresponding eigen vector by power method $A = \begin{bmatrix} 25 & 1 & 2 \\ 1 & 3 & 0 \\ 2 & 0 & -4 \end{bmatrix}$. Perform 4 iterations, taking initial guess as $[1,0,0]^T$.

MATLAB code:

```

clc;
disp(' Matrix ');
A=[25 1 2;1 3 0;2 0 -4]
disp('Initial guess');
v=[1;0;0]
disp('Number of iterations');
n=6
v0=v;
for i=1:n
v=A*v0;
M=max(v);
v=v/M;
v0=v;
fprintf('iteration number # %d\n',i)
fprintf('current eigen value is %.4f\n', M)
fprintf('current eigen value is ')
v
end

```

OUTPUT:

Matrix

A =

25	1	2
1	3	0
2	0	-4

Initial guess

v = 1

0

0

Number of iterations

n =

6

iteration number # 1

current eigen value is 25.0000

current eigen value is

v =

1.0000

0.0400

0.0800

iteration number # 2

current eigen value is 25.2000

current eigen value is

v =

1.0000

0.0444

0.0667

iteration number # 3

current eigen value is 25.1778

current eigen value is

v =

1.0000

0.0450

0.0688

iteration number # 4

current eigen value is 25.1827

current eigen value is

v =

1.0000

0.0451

0.0685

.....

PROGRAM 6: Solution of algebraic and transcendental equations by Newton-Raphson method.

Example 1: Write and execute MATLAB program to find the real root of the equation $\sin x + \cos x + e^x = 8$ which lies between 2 and 3 using Newton Raphson method.

MATLAB code:

```
f = @(x) (sin(x) + cos(x) + exp(x) - 8 );
fd = @(x) (cos(x) - sin(x) + exp(x));
p0 = input('Enter initial approximation, p0:');
n = input('Enter no. of iterations, n:');
tol = input('Enter tolerance, tol: ');
i = 1;
while i <= n
    d=f(p0)/fd(p0);
    p0 = p0 - d;
    if abs(d) < tol
        fprintf('\nApproximate root of the given equation is x%d= %0.4f\n\n',n,p0);    break;
    else
        i = i+1;
    end
end
end
```

OUTPUT:

Enter initial approximation, p0: 2

Enter no. of iterations, n: 20

Enter tolerance, tol: 0.0001

Approximate root of the given equation is x20= 2.0192

Example2: Write and execute MATLAB program to find the real root of the equation $y = \cos x$ between 0 and 1 using Newton Raphson method.

MATLAB code:

```
f = @(x) (cos(x));
fd = @(x) (-sin(x));
p0 = input('Enter initial approximaation: ');
n = input('Enter no. of iterations, n: ');
tol = input('Enter tolerance, tol: ');
i = 1;
while i <= n
    d=f(p0)/fd(p0);
    p0 = p0 - d;
    if abs(d) < tol
        fprintf('\nApproximate root of the given equation is x%d= %0.4f\n\n',n,p0);
        break;
    else
        i = i+1;
    end
end
```

OUTPUT:

```
Enter initial approximation, p0: 0
Enter no. of iterations, n: 10
Enter tolerance, tol: 0.0001
Approximate root of the given equation is x10= 1.5708
```

.....

PROGRAM 7: Solution of ODE of first order and first degree by Modified Euler's method.

Example 1: Solve $\frac{dy}{dx} = \log_e(x + y)$, $y(1) = 2$ by Modified Euler's method for $y(1.4)$ taking $h=0.1$. Display output for each value of x up to 1.4. Perform 3 modifications at every step.

MATLAB code:

```
%Euler's Modified
clear
clc
f=@(x,y) log(x+y);
x= 1:0.1:1.4;

y(1)=2;
h=0.1;
for i=2:length(x)
y(i)=y(i-1)+h*f(x(i-1),y(i-1));
y1(i,1)=y(i);
for j=2:4
y1(i,j)=y(i-1)+h/2*(f(x(i),y1(i,j-1))+f(x(i-1),y(i-1)));
end
y(i)=y1(i,4);
end
x=[x(:)];
y=[y(:)];
disp("    x    y")
z=[x y];
disp(z)
```

Output:

x	y
1.0000	2.0000
1.1000	2.1133
1.2000	2.2333
1.3000	2.3599
1.4000	2.4927

Example 2: Apply Modified Euler's method for $y(1.4)$ taking $h=0.1$. Display output for each value of x up to 1.4. Perform 3 modifications at every step.

MATLAB code:

```
clear
clc
f=@(x,y) 1+(y/x);
x= 1:0.1:1.4;
y(1)=2;
h=0.1;
for i=2:length(x)
y(i)=y(i-1)+h*f(x(i-1),y(i-1));
y1(i,1)=y(i);
for j=2:4
    y1(i,j)=y(i-1)+h/2*(f(x(i),y1(i,j-1))+f(x(i-1),y(i-1)));
end
y(i)=y1(i,4);
end
x=[x(:)];
y=[y(:)];
disp("    x    y")
z=[x y];
disp(z)
```

OUTPUT:

x	y
1.0000	2.0000
1.1000	2.3048
1.2000	2.6186
1.3000	2.9409
1.4000	3.2708

.....

PROGRAM 8: Solution of ODE of first order and first degree by 4th order Runge-Kutta method

Example 1: By using 4th order R-K method, solve $dy/dx=xy^{1/3}$, $y(1)=1$ at $x=1.5$ by taking step size as 0.1.

MATLAB code:

```
clear all;
clc;
f = @(x,y)x.*(y.^(1/3)) ;
x = 1; %initial value of x
y = 1; %initial value of y(x)
h = 0.1; %step size
X = 1.5; %y(1.5)
%X-x
while X-x >= -10^(-10) %smaller value
    fprintf("value of y at x = %0.1f is %0.4f\n",x,y);

    k1 = h.*f(x,y);
    k2 = h.*f(x+h/2,y+k1/2);
    k3 = h.*f(x+h/2,y+k2/2);
    k4 = h.*f(x+h, y+k3);
    k = (k1 + 2.* k2 +2.*k3+k4 )./6;
    x =x+h;
    y = y+k;
end
```

OUTPUT:

```
value of y at x = 1.0 is 1.0000
value of y at x = 1.1 is 1.1068
value of y at x = 1.2 is 1.2279
value of y at x = 1.3 is 1.3641
value of y at x = 1.4 is 1.5166
value of y at x = 1.5 is 1.6862
```

Example 2: By using 4th order R-K method, solve $dy/dx = 3x + \frac{y}{2}$, $y(0) = 1$ at $x = 0.2$ by taking step size as 0.1.

MATLAB code:

```
clear all;
clc;
f = @(x,y)(3*x+(y/2)) ;
x = 0; %initial value of x
y = 1; %initial value of y(x)
h = 0.1; %step size
X = 0.2; %y(1.5)
%X-x
while X-x >= -10^(-10) %smaller value
    fprintf("value of y at x = %0.1f is %0.4f\n",x,y);
    k1 = h.*f(x,y);
    k2 = h.*f(x+h/2,y+k1/2);
    k3 = h.*f(x+h/2,y+k2/2);
    k4 = h.*f(x+h, y+k3);
    k = (k1 + 2.* k2 +2.*k3+k4 )./6;
    x =x+h;
    y = y+k;
end
```

OUTPUT:

```
value of y at x = 0.0 is 1.0000
value of y at x = 0.1 is 1.0665
value of y at x = 0.2 is 1.1672
```

.....

PROGRAM 9: Interpolation/Extrapolation using Newton's forward and backward difference formulae.

NEWTON'S FORWARD INTERPOLATION FORMULA

Example 1: Write the MATLAB code that uses Newton's forward interpolation formula and find the approximate value of $f(9)$ given the following data points $(x,y) = (8,10), (10,19), (12,32.5), (14,54), (16,89.5) \text{ \& } (18,154)$.

MATLAB code:

```
%Newton's Forward Difference Formula MATLAB Program
```

```
clc;
```

```
clear all;
```

```
x=[8,10,12,14,16,18];
```

```
y=[10,19,32.5,54,89.5,154];
```

```
n=length(x);
```

```
X=9;
```

```
h=x(2)-x(1);
```

```
F=zeros(n,n);
```

```
F(:,1)=y;
```

```
for j=2:n
```

```
    for i=j:n
```

```
        F(i,j)=F(i,j-1)-F(i-1,j-1);
```

```
    end
```

```
end
```

```
p=(X-x(1))/h;
```

```
d=F(1,1)+p*F(2,2)+p*(p-1)*F(3,3)/factorial(2)+p*(p-1)*(p-2)*F(4,4)/factorial(3)+p*(p-1)*(p-2)*(p-3)*F(5,5)/factorial(4)+p*(p-1)*(p-2)*(p-3)*(p-4)*F(6,6)/factorial(5);
```

```
fprintf('f(%0.4f)= %0.4f\n', X, d);
```

Output:

f(9.0000)= 14.2363

NEWTON'S BACKWARD INTERPOLATION FORMULA

Example 2: Write the MATLAB code that uses Newton's backward interpolation formula and find the approximate value of $f(17)$ given the following data points $(x,y) = (8,10), (10,19), (12,32.5), (14,54), (16,89.5) \text{ \& } (18,154)$.

MATLAB code:

```
clc;
clear all;
x=[8,10,12,14,16,18];
y=[10,19,32.5,54,89.5,154];
n=length(x);
X=17;
h=x(2)-x(1);
F=zeros(n,n);
F(:,1)=y;
for j=2:n
    for i=j:n
        F(i,j)=F(i,j-1)-F(i-1,j-1);
    end
end
p=(X-x(n))/h;
d=F(6,1)+p*F(6,2)+p*(p+1)*F(6,3)/factorial(2)+p*(p+1)*(p+2)*F(6,4)/factorial(3)+p*(p+1)*(p+2)*(p+3)*F(6,5)/factorial(4)+p*(p+1)*(p+2)*(p+3)*(p+4)*F(6,6)/factorial(5);
fprintf('f(%0.4f)= %0.4f\n', X, d);
```

Output:

f(17.0000)= 116.6582

.....

PROGRAM 10: Computation of area under the curve using Trapezoidal,Simpson's (1/3)rd and (3/8)th rules.

Example 1: Evaluate $\int_0^3 \sqrt{4+x^3} dx$ by using Trapezoidal Rule by taking no. of sub-intervals as 6.

MATLAB code:

```
clc;
x = 0:0.5:3;
y = sqrt(4+x.^3);
fprintf('The value of the Integral is given by')
I = trapz(x,y)
```

Output:

The value of integration is 9.3304

Example 2: Evaluate $\int_0^1 \sqrt{(\sin(x) + \cos(x))} dx$ by using **Simpson's 1/3rd** Rule.

MATLAB code:

```
clear all;
clc;
f=@(x)(sqrt(sin(x)+cos(x)));
a=input('Enter the lower limit a:');
b=input('Enter the upper limit b:');
n=input('Enter the number of sub-intervals n:');
h=(b-a)/n;
for i=1:1:n-1
y(i)=f(a+i.*h);
end
s_o=0;
s_e=0;
for i=1:1:n-1
    if rem(i,2)==0
        s_e=s_e+y(i);
    else
        s_o=s_o+y(i);
    end
end
I=(h/3)*(f(a)+f(b)+2*s_e+4*s_o);
fprintf('\n The value of integration is %.4f\n',I);
```

Output:

```
Enter the lower limit a:
0
Enter the upper limit b:
1
Enter the number of sub-intervals n:
6
The value of integration is 1.1394
```

.....